Cloud File Deduplication & Storage Optimizer

D. Y. Patil College of Engineering & Technology, Kolhapur | Department of CSE | Batch T6 (2026-27)

Institution: D. Y. Patil College of Engg. & Tech., Kolhapur	**Course:** Cloud Computing (CC) Laboratory
Department: Computer Science & Engineering	**Batch / Academic Year:** Batch T6 (2026 – 2027)
Project Guide: Mrs. Aditee A. Salokhe	**Head of Department:** Dr. R. J. Dhanal
Live Stack: Netlify (Frontend) | Render (Backend)	**Database & Storage:** Supabase PostgreSQL + S3 Storage

Roll No.	Student Name (T.Y. CSE - Div C)	Project Module & Responsibility
155	Prasanna Pradeep Bhurke	Cloud Backend Architecture, Supabase DB & Storage Integration, Live Tunnels
144	Sarthak Sanjay Salokhe	Block Hashing Engine (SHA-256), Deduplication Algorithms
149	Vyanktesh Harishchandra Madhyapgol	Frontend Glassmorphic UI & Visual Block Distribution Map
151	Manjunath Rajkumar Gavandi	Storage Analytics, Space Savings Calculator & Reporting
152	Balaji Ramraje Gaikwad	Testing Suite, File Reconstruction Pipeline & Validation

1. Executive Overview & Problem Statement

Modern cloud storage systems face severe data redundancy. Studies indicate that **40% to 70%** of data stored in organizational clouds (Google Drive, Dropbox, AWS S3) consists of duplicate files, minor revisions, and identical shared attachments. Storing duplicate bytes wastes cloud budgets, network bandwidth, and backup infrastructure.

The **Cloud File Deduplication & Storage Optimizer** is an enterprise-grade cloud system that eliminates redundant file storage. By slicing files into uniform **64 KB chunk blocks**, generating cryptographically collision-resistant **SHA-256 fingerprints**, and managing reference counts within a central metadata catalog, the system guarantees that **any identical chunk is stored physically only once** across the entire cloud.

2. Full Technology Stack: Frontend, Backend & Database

Frontend (UI/UX)	**HTML5, Modern CSS3, JavaScript (Vanilla ES6+), Netlify CDN**	• Responsive Glassmorphism dashboard with dark mode palette. • Drag-and-drop file upload zone with real-time chunk visualization. • Live analytics cards: deduplication ratio, percentage reduction, logical vs. physical space. • Visual chunk map showing shared vs. unique blocks. • Hosted globally on Netlify CDN with automated API proxy rewrites.
Backend (Engine)	**Python 3, Flask, Gunicorn, REST API, Render Web Service**	• Slices file streams into uniform 64 KB binary blocks dynamically. • Calculates SHA-256 digests in linear O(N) streaming time. • Cross-Origin Resource Sharing (CORS) enabled for microservice communication. • Asynchronous chunk verification & reference counting. • On-demand file reconstruction and chunk streaming for byte-perfect downloads. • Deployed 24/7 on Render cloud with automated auto-scaling.
Database (Metadata)	**Supabase Cloud PostgreSQL + IPv4 Connection Pooler**	• Relational schema with ACID compliance. • Foreign key cascades linking logical files to physical chunk hashes. • Indexed lookups on 64-character SHA-256 fingerprints for O(1) duplicate checks. • Resilient database adapter with automatic fallback to local SQLite if offline.
Storage (Object Layer)	**Supabase Cloud Storage (S3-Compatible Bucket)**	• Stores unique raw binary chunks in the 'cloud-chunks' bucket. • Local LRU binary caching to accelerate read throughput and reconstruction. • Seamless dual-adapter mode: Cloud Object Storage vs. Local directory bucket.

3. Chunk-Level Deduplication Algorithm & Lifecycle

Deduplication occurs at the **chunk level** (sub-file level) rather than whole-file level. This ensures that when a user uploads an edited version of a 100 MB file where only 1 paragraph was changed, 99.9% of the unchanged chunks are reused, saving massive cloud storage.

- **Step 1 — Stream Ingestion:** The uploaded file is read as an in-memory binary stream without saving unverified temporary files to disk.
- **Step 2 — Fixed-Size Chunking:** The stream is partitioned into discrete 64 KB (65,536 bytes) binary blocks.
- **Step 3 — Cryptographic Hashing:** For each chunk, a SHA-256 cryptographic fingerprint (a 64-character hexadecimal digest) is generated.
- **Step 4 — Catalog Hash Lookup:** The engine queries PostgreSQL: *SELECT ref_count FROM chunks WHERE chunk_hash = %s*.
- **Step 5 — Conditional Action:**
  - **If Hash Exists (Duplicate):** Skip physical upload! Increment chunk *ref_count = ref_count + 1* in database.
  - **If Hash is New:** Upload 64 KB payload to Supabase Object Storage, insert new record in *chunks* with *ref_count = 1*.
- **Step 6 — Virtual File Registration:** Insert the logical file record into *user_files* and map each chunk position into *file_chunks(file_id, chunk_index, chunk_hash)*.

4. Relational Database Schema & Data Models

The relational schema is engineered to decouple physical cloud chunks from virtual user files:

Table	Key Columns & Types	Purpose
chunks (Physical Catalog)	• chunk_hash (VARCHAR 64, PK) • size_bytes (BIGINT) • ref_count (INTEGER) • first_uploaded_at (TIMESTAMPTZ)	Stores the catalog of unique physical binary chunks in cloud storage and tracks active references across all files.
user_files (Virtual Metadata)	• id (BIGSERIAL, PK) • filename (TEXT) • user_name (TEXT) • file_size (BIGINT) • file_hash (VARCHAR 64) • total_chunks, dedup_chunks, new_chunks • upload_timestamp (TIMESTAMPTZ) • is_deduplicated (BOOLEAN)	Represents the logical files visible to users on the dashboard, tracking deduplication efficiency metrics per file.
file_chunks (Reassembly Map)	• id (BIGSERIAL, PK) • file_id (BIGINT, FK -> user_files.id) • chunk_index (INTEGER) • chunk_hash (VARCHAR 64, FK -> chunks)	The reassembly recipe! Maintains the exact sequential order of chunk hashes required to reconstruct any file.

5. File Reconstruction & Byte-Perfect Streaming Pipeline

When a user clicks **Download** for file ID *k*, the system dynamically reconstructs the original file on-the-fly without keeping pre-built duplicates on disk:

- **Ordered Chunk Query:** The engine queries *file_chunks* ordered by *chunk_index ASC* to obtain the exact sequential list of hashes.
- **Chunk Retrieval:** For each hash, the storage adapter checks its local cache first (hit time < 1ms); if missing, it fetches the chunk binary from Supabase Object Storage.
- **Generator Stream Pipeline:** Chunks are yielded sequentially through Python's WSGI generator protocol, piped directly into Flask's HTTP *Response(stream, mimetype='application/octet-stream')*.
- **Zero Memory Bloat:** Large multi-gigabyte files can be streamed to the client with fixed, constant RAM consumption (64 KB working memory).

6. Production Cloud Deployment Architecture

The project is engineered as a decoupled, multi-cloud microservices architecture:

Frontend SPA	**Netlify**	Static site hosting on high-speed global edge CDN. Proxy redirect in netlify.toml forwards /api/* to backend seamlessly with zero CORS friction.
Backend API	**Render**	Continuous delivery from GitHub repository. Runs production Gunicorn WSGI workers on Linux container with Python 3.14.
Relational DB	**Supabase (PostgreSQL)**	Managed PostgreSQL with Supavisor connection pooler on port 6543 (IPv4 compatible for Render environments).
Object Storage	**Supabase Storage**	S3-compatible bucket cloud-chunks storing deduplicated binary payload blocks with cryptographic naming.
Local Dev Tunnel	**Cloudflare Tunnel**	Instant public SSL URL for offline demonstrations via run_all_live.bat.

7. Storage Analytics & Space Optimization Formulas

The analytics module continuously computes live metrics across all cloud storage operations:

- **Total Logical Size:** $L = \Sigma\ file_size_i$ (Total data size users believe is stored).
- **Total Physical Size:** $P = \Sigma\ size_bytes_j$ (Actual unique chunks stored in Supabase).
- **Space Saved:** $S = L - P$ (Bytes eliminated from physical cloud disk).
- **Deduplication Ratio:** $R = L / P : 1$ (e.g., 5.4 : 1 means 5.4x storage efficiency).
- **Savings Percentage:** $\% = ((L - P) / L) \times 100\%$ (e.g., 81.5% storage cost reduction).

8. Security, Data Integrity & Crash-Resilient Architecture

• **Collision Resistance:** SHA-256 probability of hash collision is less than 1 in 10^{77}, statistically impossible in practical cloud systems.
• **Graceful DB Fallback:** In [core/db.py], if cloud PostgreSQL experiences network latency or connection pooling delays, the system automatically falls back to SQLite so the web service never crashes.
• **Safe Deletion (Reference Counting):** Deleting a file decrements chunk reference counts. A physical chunk is only purged from cloud storage when its *ref_count == 0*, preventing accidental data loss.